

Banker's Algorithm

Deadlock Avoidance in Operating Systems

Subject:	Operating System
Course:	ICE 2247
Submitted To:	Md. Azizul Hakim Lecturer Dept. of Information and Communication Engineering Daffodil International University
Level-Term:	L2-T2

1. Introduction

In modern operating systems, multiple processes simultaneously compete for limited resources such as CPU time, memory, files, and I/O devices. Improper or uncontrolled allocation of these shared resources can lead to a critical system condition known as deadlock, in which processes wait indefinitely and the system comes to a standstill.

To address this problem, the renowned computer scientist Edsger W. Dijkstra introduced the Banker's Algorithm — a well-known deadlock avoidance technique. The algorithm is named "banker's" because it mirrors the behavior of a cautious bank that only grants loans (resources) when it can guarantee that all future transactions (process executions) can still be completed safely.

Core Principle: Before allocating any resource to a process, the system first checks whether the resulting state will remain safe — i.e., whether all processes can still complete their execution.

2. Deadlock Overview

2.1 What is Deadlock?

A deadlock is a situation in an operating system where a set of processes are permanently blocked because each process is waiting for a resource that is currently held by another process in the same set. None of the processes can proceed, and the system effectively freezes.

2.2 Four Necessary Conditions for Deadlock

Deadlock can only occur if all four of the following conditions hold simultaneously (Coffman Conditions, 1971):

Mutual Exclusion: At least one resource must be held in a non-shareable mode; only one process can use the resource at a time.

Hold and Wait: A process holding at least one resource is waiting to acquire additional resources held by other processes.

No Preemption: Resources cannot be forcibly taken away from a process; they must be voluntarily released.

Circular Wait: A circular chain of two or more processes exists, where each process is waiting for a resource held by the next process in the chain.

The Banker's Algorithm works specifically by preventing Circular Wait through careful safe-state analysis before each resource allocation.

3. Banker's Algorithm — Core Concept

3.1 Safe State

A system is said to be in a safe state if there exists at least one safe sequence — an ordering of all processes — such that each process in the sequence can obtain the resources it needs using the currently available resources plus the resources released by the processes that precede it in the sequence.

Example of a safe sequence: $\langle P1, P3, P0, P2 \rangle$

3.2 Unsafe State

An unsafe state is one in which no safe sequence exists. In an unsafe state, there is a possibility (though not a certainty) that the system will eventually deadlock. The Banker's Algorithm prevents the system from ever entering an unsafe state by denying any resource request that would lead to one.

Important Note: An unsafe state does not necessarily mean deadlock has occurred; it means deadlock may occur. The Banker's Algorithm conservatively avoids all unsafe states.

4. Data Structures Used

Let n = number of processes and m = number of resource types. The Banker's Algorithm uses the following data structures:

Available[j]: The number of currently available instances of resource type j .

Max[i][j]: The maximum number of instances of resource type j that process i may ever request.

Allocation[i][j]: The number of instances of resource type j currently allocated to process i .

Need[i][j]: The remaining resource need of process i for resource type j .

$$\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$$

5. Banker's Algorithm — Detailed Steps

5.1 Safety Algorithm

The Safety Algorithm is the core engine of the Banker's Algorithm. Given the current system state — the Available vector, the Allocation matrix, and the Need matrix — it determines whether there exists at least one safe execution sequence for all processes. If such a sequence can be constructed, the system is declared safe and no deadlock can occur; otherwise it is declared unsafe.

Core Purpose: Verify, without making any real allocation change, whether every process in the system can eventually obtain the resources it needs and terminate.

Input & Output

Input	Available[], Allocation[][], Need[][]
Output	SAFE (with a safe sequence printed) OR UNSAFE

Variables Used in the Algorithm

Variable	Initial Value	Role in the Algorithm
Work[]	= Available[]	Simulates available resources as processes "finish". Grows as each process releases its allocation.
Finish[i]	= False for all i	Tracks which processes have been virtually completed. True means Pi can safely finish.
Safe Sequence	= [] (empty list)	Records the order in which processes are confirmed to be able to finish. Output of the algorithm.

Detailed Step-by-Step Explanation

Step 1 — Initialization

Before the loop begins, set up Work and Finish:

Work[j] ← Available[j]	for all j in {0 ... m-1}
Finish[i] ← False	for all i in {0 ... n-1}

Work is a working copy of the available resource vector. It will grow as processes are virtually completed. Finish[i] = False means we have not yet established that Pi can run to completion.

Step 2 — Find an Eligible (Runnable) Process

Scan all processes and find any process P_i satisfying both conditions simultaneously:

- $\text{Finish}[i] == \text{False} \rightarrow P_i$ has not yet been scheduled in the sequence
- $\text{Need}[i][j] \leq \text{Work}[j]$ for ALL $j \rightarrow P_i$'s remaining resource needs can be met with what is currently available

The second condition is checked component-wise: every single resource type must satisfy the inequality. If even one resource type fails the check, P_i is not eligible in this iteration.

Intuition: We are asking “Is there any process that could safely run right now if we gave it what it still needs?” If yes, we let it run (virtually). If no process can run, the system is stuck — unsafe.

Example: $\text{Work} = (3,3,2)$, $\text{Need}[P1] = (1,2,2)$. Check: $1 \leq 3?$ ✓ $2 \leq 3?$ ✓ $2 \leq 2?$ ✓
 $\rightarrow P1$ is eligible.

Step 3 — Simulate Process Completion (Resource Release)

If an eligible process P_i is found, simulate it running to completion:

<pre>Work[j] ← Work[j] + Allocation[i][j] for all j Finish[i] ← True SafeSeq.append(P_i)</pre>

When a process finishes, it releases all the resources it was holding ($\text{Allocation}[i]$). Adding these back to Work models this release, making more resources available for other processes.

After this update, return to Step 2 and search for the next eligible process using the now-larger Work vector.

Step 4 — Repeat Until No Eligible Process Remains

Steps 2 and 3 are executed in a loop. Each full pass either adds one new process to the Safe Sequence (if one is found) or exits the loop (if none is found). This loop terminates when:

- All n processes have been added to Safe Sequence (best case), OR
- No unfinished process has its $\text{Need} \leq \text{Work}$ (deadlock scenario).

Important: different eligible processes may be found in different orders during different runs or with different starting states. All valid orderings are acceptable safe sequences.

Step 5 — Final Decision: Safe or Unsafe

After the loop exits, examine the Finish array:

```
IF     $\forall i$ : Finish[i] == True    (every process finished virtually)
    THEN  $\rightarrow$  System is SAFE | Output: Safe Sequence
ELSE  (at least one Finish[i] == False)
    THEN  $\rightarrow$  System is UNSAFE | Potential deadlock exists
```

If any Finish[i] remains False, that process could never be scheduled — it is stuck waiting for resources that will never become available under the current allocation. This signals a potentially deadlocked system.

Complete Pseudocode

```
SAFETY ALGORITHM (Banker's Algorithm)



---


INPUT : n processes, m resource types
        Available[m], Allocation[n][m], Need[n][m]
OUTPUT: Safe Sequence OR 'UNSAFE'

Step 1: Work[0..m-1]  $\leftarrow$  Available[0..m-1]
        Finish[0..n-1]  $\leftarrow$  {False, False, ..., False}
        SafeSeq  $\leftarrow$  [ ]

Step 2: REPEAT:
        found  $\leftarrow$  False
        FOR i  $\leftarrow$  0 TO n-1:
            IF Finish[i] = False AND Need[i][*]  $\leq$  Work[*]:
                Work  $\leftarrow$  Work + Allocation[i]
                Finish[i]  $\leftarrow$  True
                SafeSeq.append(Pi)
                found  $\leftarrow$  True
        IF NOT found: EXIT LOOP

Step 3: IF ( $\forall i$ : Finish[i] = True):
        RETURN 'SAFE', SafeSeq
ELSE:
        RETURN 'UNSAFE'
```

Execution Trace — Numerical Example (Available = 3,3,2)

The following table traces every iteration of the Safety Algorithm for the example in Section 6 (P0–P3, Resources A/B/C):

Iter.	Process Checked (condition met?)	Work after release	Finish[] update	Safe Seq. so far
1	P1: $\text{Need}(1,2,2) \leq \text{Work}(3,3,2)$ ✓	$(3,3,2) + (2,0,0) = (5,3,2)$	$\text{Finish}[1] = T$	P1
2	P3: $\text{Need}(0,1,1) \leq \text{Work}(5,3,2)$ ✓	$(5,3,2) + (2,1,1) = (7,4,3)$	$\text{Finish}[3] = T$	P3
3	P0: $\text{Need}(7,4,3) \leq \text{Work}(7,4,3)$ ✓	$(7,4,3) + (0,1,0) = (7,5,3)$	$\text{Finish}[0] = T$	P0
4	P2: $\text{Need}(6,0,0) \leq \text{Work}(7,5,3)$ ✓	$(7,5,3) + (3,0,2) = (10,5,5)$	$\text{Finish}[2] = T$	P2
5	No unfinished process with $\text{Need} \leq \text{Work}$	—	—	Loop exits

✓ All $\text{Finish}[i] = \text{True} \rightarrow$ System is SAFE | Safe Sequence: $\langle P1, P3, P0, P2 \rangle$

Key Points to Remember

- No real allocation is changed — the algorithm only simulates virtual completions to test whether safety can be guaranteed.
- Work grows monotonically: once a process “releases” its allocation into Work, that vector never shrinks again during the same run.
- The safe sequence is not unique. Multiple valid sequences may exist; any one of them is sufficient proof that the system is safe.
- Worst-case time complexity is $O(n^2 \times m)$: for each of n processes added, the algorithm may scan all n processes across m resource types.
- UNSAFE $\neq$ deadlock has occurred. It means deadlock ‘may’ occur if processes proceed with their maximum demands — the system is merely in a risky state.

5.2 Resource-Request Algorithm

When process P_i requests resources ($\text{Request}[i]$):

- Step 1: If $\text{Request}[i] > \text{Need}[i] \rightarrow$ Error: process exceeds its maximum claim.
- Step 2: If $\text{Request}[i] > \text{Available} \rightarrow P_i$ must wait (resources not available).
- Step 3: Tentatively allocate: $\text{Available} -= \text{Request}[i]$; $\text{Allocation}[i] += \text{Request}[i]$; $\text{Need}[i] -= \text{Request}[i]$.
- Step 4: Run the Safety Algorithm. If safe $\rightarrow$ grant request. If unsafe $\rightarrow$ rollback and make P_i wait.

6. Numerical Example (Step-by-Step)

Given: Processes P0, P1, P2, P3 and Resources A, B, C. Available = (3, 3, 2).

Allocation Matrix

Process	A	B	C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1

Max Matrix

Process	A	B	C
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2

Need Matrix (Need = Max – Allocation)

Process	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1

6.1 Safe Sequence Derivation

Step	Process Check	Updated Work
Initial State	Work = (3, 3, 2)	
Step 1	P1: Need=(1,2,2) ≤ Work=(3,3,2) ✓	Work = (3,3,2)+(2,0,0) = (5,3,2)
Step 2	P3: Need=(0,1,1) ≤ Work=(5,3,2) ✓	Work = (5,3,2)+(2,1,1) = (7,4,3)
Step 3	P0: Need=(7,4,3) ≤ Work=(7,4,3) ✓	Work = (7,4,3)+(0,1,0) = (7,5,3)
Step 4	P2: Need=(6,0,0) ≤ Work=(7,5,3) ✓	Work = (7,5,3)+(3,0,2) = (10,5,5)

✓ Safe Sequence: < P1, P3, P0, P2> — System is in a SAFE STATE

7. Python Implementation

The following Python program implements the Banker's Safety Algorithm. It takes allocation, maximum demand, and available resource matrices as input and determines whether the system is in a safe state.

```
def bankers_algorithm(allocation, max_demand, available):
    n = len(allocation)      # Number of processes
    m = len(available)       # Number of resource types

    # Step 1: Calculate Need matrix
    need = [[max_demand[i][j] - allocation[i][j]
              for j in range(m)] for i in range(n)]

    print('Need Matrix:')
    for i, row in enumerate(need):
        print(f' P{i}: {row}')

    finish = [False] * n
    work = available.copy()
    safe_sequence = []

    # Step 2-4: Find safe sequence
    while len(safe_sequence) < n:
        found = False
        for i in range(n):
            if not finish[i] and all(need[i][j] <= work[j] for j in
range(m)):
                work = [work[j] + allocation[i][j] for j in
range(m)]
                safe_sequence.append(f'P{i}')
                finish[i] = True
                found = True
        if not found:
            print('System is NOT SAFE (Deadlock possible)')
            return

    print('\nSystem is SAFE')
    print('Safe Sequence:', ' ' -> ' '.join(safe_sequence))

# --- Input Data ---
allocation = [[0,1,0], [2,0,0], [3,0,2], [2,1,1]]
max_demand = [[7,5,3], [3,2,2], [9,0,2], [2,2,2]]
available = [3, 3, 2]
```

```
bankers_algorithm(allocation, max_demand, available)
```

7.1 Sample Output

Need Matrix:

P0: [7, 4, 3]

P1: [1, 2, 2]

P2: [6, 0, 0]

P3: [0, 1, 1]

System is SAFE

Safe Sequence: P1 -> P3 -> P0 -> P2

8. Time and Space Complexity

Metric	Complexity	Explanation
Time Complexity	$O(n^2 \times m)$	The outer while-loop runs at most n times; the inner for-loop runs n times; each iteration checks m resources.
Space Complexity	$O(n \times m)$	Storage for Allocation, Max, and Need matrices, each of size $n \times m$.

9. Advantages and Disadvantages

9.1 Advantages

- Completely avoids deadlock by ensuring the system always stays in a safe state.
- Provides a systematic, mathematical approach to resource allocation.
- Allows multiple processes to coexist without interfering with each other's progress.
- Particularly useful in batch-processing systems with predictable resource demands.

9.2 Disadvantages

- Requires each process to declare its maximum resource demand in advance, which is often impractical.
- Not suitable for real-time or interactive systems due to computational overhead.

- Assumes a fixed number of processes and resources; dynamic systems are not well-supported.
- May lead to under-utilization of resources by being overly conservative.

10. Real-Life Analogy

The Banker's Algorithm gets its name from the way a prudent bank manages loans. Consider the following analogy:

Banking Concept	OS Concept
Bank	Operating System
Customers	Processes
Money (loan amount)	Resources
Maximum loan request	Max matrix
Current loan outstanding	Allocation matrix
Remaining loan needed	Need matrix
Bank's current cash reserve	Available vector

Just as a bank only approves a loan if it can still guarantee that all its customers can eventually be fully repaid, the operating system only grants a resource request if it can guarantee all processes will eventually complete — keeping the system deadlock-free.

11. Conclusion

The Banker's Algorithm is a fundamental and elegant solution to the problem of deadlock avoidance in operating systems. By maintaining a careful accounting of resource allocations, maximum demands, and remaining needs, it enables the system to make proactive decisions that guarantee progress for all processes.

The key strength of the algorithm lies in its safety-check mechanism: before granting any resource request, the system simulates the allocation and verifies that a safe sequence still exists. If no such sequence can be found, the request is deferred, preventing the system from entering an unsafe state.

While its practical limitations — such as the need to know maximum demands in advance and high computational overhead — restrict its direct deployment in general-purpose systems, the Banker's Algorithm remains an indispensable theoretical model for understanding deadlock, resource management, and safe concurrency. It continues to be a cornerstone topic in operating systems education and research.

References

- [1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). John Wiley & Sons.
- [2] Tanenbaum, A. S., & Bos, H. (2014). Modern Operating Systems (4th ed.). Pearson.
- [3] Dijkstra, E. W. (1965). Solution of a Problem in Concurrent Programming Control. Communications of the ACM, 8(9), 569.
- [4] Coffman, E. G., Elphick, M., & Shoshani, A. (1971). System Deadlocks. ACM Computing Surveys, 3(2), 67–78.